LinkFlow Project Guide

1. What Is LinkFlow?

LinkFlow is a Windows desktop productivity app for saving links, files, folders, PDFs, clipboard text, and meeting references in one place.

The app stays available in the background through a small floating widget and a system tray icon. The main idea is simple: capture something quickly without opening Notepad or losing it in clipboard history.

2. Main Parts Of The Project

Landing Page

The ‘landing‘ folder contains the public product website. It explains LinkFlow and includes a Windows download button.

Desktop App

The ‘desktop‘ folder contains the Electron Windows application. It controls the background widget, tray menu, shortcut, capture actions, notifications, and local storage.

Assets

The ‘assets‘ folder contains the shared SVG and ICO logos used by the website, tray, and Windows installer.

Scripts

The ‘scripts‘ folder contains tools for creating the Windows icon and building the installer.

3. Important Files

â¢ ‘desktop/main.js‘: Electron main process and background behavior.
â¢ ‘desktop/preload.js‘: Safe bridge between the widget and Electron.
â¢ ‘desktop/widget.html‘: Floating capture widget UI.
â¢ ‘desktop/electron-builder.json‘: Windows installer and GitHub publishing configuration.
â¢ ‘landing/index.html‘: Landing page structure and Tailwind classes.
â¢ ‘package.json‘: Project scripts and dependencies.
â¢ ‘.github/workflows/release.yml‘: Automatic Windows release workflow.
â¢ ‘vercel.json‘: Static Vercel deployment configuration.

4. How The Desktop App Works

When LinkFlow starts, Electron creates a transparent always-on-top window. The window loads ‘widget.html‘ and places the widget near the bottom-right of the primary monitor.

The application also creates a system tray icon. The global shortcut ‘Ctrl + Shift + L‘ shows or hides the widget.

When a user captures an item, the widget sends the data through the preload bridge. The main process saves it to a local ‘items.json‘ file inside Electron’s user-data directory and shows a

The current MVP supports clipboard text, URLs, files, folders, and drag-and-drop file capture.

5. How To Run The Project

Install Node.js 20 or newer, then install dependencies from the project folder:

```
npm install
```

Run the landing page:

```
npm run dev:landing
```

Open ‘http://localhost:4173‘.

Run the desktop app:

```
npm run dev:desktop
```

Press ‘Ctrl + Shift + L‘ to show or hide the widget.

6. How To Test Capture

Clipboard URL

1. Copy a URL such as ‘https://github.com‘.
2. Open LinkFlow.
3. Click the plus button.
4. A saved notification should appear.

File Capture

1. Open File Explorer.
2. Drag a file onto the widget.
3. LinkFlow saves the file path locally.

File Picker

1. Click the file button.
2. Select a file or folder.
3. LinkFlow saves the selected path.

7. Building The Windows Installer

The installer is created in the parent ‘dist‘ folder and copied to:

landing\downloads\LinkFlow-Setup.exe

The installer is unsigned during development. Windows SmartScreen may display a warning for new unsigned applications.

8. Publishing On GitHub

The GitHub repository is:

https://github.com/hemanth174/LinkFlow

The repository must contain the project source code and the release workflow. To publish a version, create a version tag:

```
npm version patch
git push origin main --follow-tags
```

The GitHub Action builds the Windows installer on a Windows runner and publishes it to GitHub Releases.

The landing page downloads the latest installer from:

https://github.com/hemanth174/LinkFlow/releases/latest/download/LinkFlow-Setup.exe

That URL works only after a GitHub Release contains an asset with exactly that filename.

9. How Updates Work

Every desktop update must have a new version number. For example:

â¢ ‘0.1.0‘ becomes ‘0.1.1‘ for a patch.
â¢ ‘0.1.0‘ becomes ‘0.2.0‘ for a feature release.
â¢ ‘0.1.0‘ becomes ‘1.0.0‘ for a major release.

Packaged LinkFlow applications use ‘electron-updater‘ to check GitHub Releases. When a new release is downloaded, the user receives a notification and the update is installed after restart.

Development mode does not receive production updates.

10. Deploying The Landing Page

The landing page is a static website and can be deployed with Vercel.

Use these Vercel settings:

Framework Preset: Other
Build Command: empty
Output Directory: landing
Install Command: npm install

The repository's 'vercel.json' contains the same settings. After pushing changes to 'main', Vercel automatically creates a new deployment.

11. Common Problems

Python Was Not Found

The project does not need Python. Use 'npm run dev:landing' after running 'npm install'.

Electron Is Not Recognized

Run:

```
npm install
```

Then run 'npm run dev:desktop'.

Download Shows 404

GitHub does not have a release asset named 'LinkFlow-Setup.exe'. Publish a tagged release and confirm the asset filename.

Vercel Says Output Directory Is Missing

Set the Output Directory to 'landing' and leave the Build Command empty. Remove old Production Overrides if they still contain another setting.

SmartScreen Warning

The installer is unsigned or has low reputation. A trusted Windows code-signing certificate is required for professional distribution.

12. Future Features

â¢ Saved-items dashboard
â¢ Collections and tags
â¢ Search
â¢ PDF and website previews
â¢ Meeting scheduling
â¢ Ten-minute reminders
â¢ Broken-path relinking

â¢ Settings and notification preferences
â¢ Browser extension
â¢ Cloud synchronization

13. Learning Path

Learn the project in this order:

1. Read ‘package.json‘ to understand commands.
2. Read ‘desktop/main.js‘ to understand Electron.
3. Read ‘desktop/preload.js‘ to understand the safe IPC bridge.
4. Read ‘desktop/widget.html‘ to understand capture interactions.
5. Read ‘landing/index.html‘ to understand the website.
6. Read ‘desktop/electron-builder.json‘ to understand packaging.
7. Read ‘.github/workflows/release.yml‘ to understand deployment autor